TRAZA

Manual del lenguaje

Cómo escribir pseudocódigo, leer el diagrama de flujo, armar los bloques y traducir el algoritmo

Este manual es para quien va a crear algoritmos. Explica cada instrucción, qué datos hay que poner, las palabras reservadas, las figuras del diagrama, los bloques del laboratorio y las funciones incluidas, con un ejemplo en cada caso. El último capítulo explica el botón Traducir: genera el mismo algoritmo en otros lenguajes y no lo ejecuta.

19 de octubre de 2026

1. Cómo está armado un algoritmo

Un archivo tiene un solo algoritmo. Puede tener funciones y procedimientos antes o después, nunca dentro. El nombre del algoritmo es un identificador: empieza con letra o guion bajo y no puede ser una palabra reservada.



Figura de inicio. Todo diagrama empieza en un óvalo.

```
Algoritmo Saludar

Definir nombre Como Cadena;

Escribir "¿Cómo te llamas?";

Leer nombre;

Escribir "Hola, ", nombre;

FinAlgoritmo
```



Figura de fin. El óvalo cierra el algoritmo o la subrutina.

- La asignación se escribe con <-. El signo = compara y no guarda un valor.
- En el perfil estricto, que es el de la aplicación, cada sentencia simple termina en punto y coma. FinSi, FinMientras, FinPara, FinSegun, FinFuncion y FinAlgoritmo no llevan punto y coma.
- Un algoritmo puede desactivar el punto y coma o hacer que los arreglos empiecen en 0. Eso se cambia en Configuración y afecta a ese algoritmo.
- Los comentarios de una línea empiezan con //. Los de varias líneas van entre /* y */ y no se anidan.
- Las palabras reservadas no distinguen mayúsculas ni acentos. Los nombres de variables sí: año y ano son distintas.
- Si la cuenta está en inglés, el editor muestra Algorithm, If o Write. El programa es el mismo. Este manual escribe las palabras en español.

2. Palabras reservadas

Estas palabras ya tienen un significado. No se pueden usar como nombre de variable, de algoritmo ni de función. La columna de la derecha es la grafía que aparece cuando el idioma de la cuenta es inglés.

Para qué	En español	En inglés
Programa	Algoritmo, FinAlgoritmo	Algorithm, EndAlgorithm
Función	Función, FinFunción	Function, EndFunction
Procedimiento	SubProceso, FinSubProceso	SubProcedure, EndSubProcedure
Declarar	Definir, Como	Declare, As
Arreglo	Dimension	Dimension
Parámetros	Por, Valor, Referencia	By, Value, Reference
Entrada y salida	Leer, Escribir	Read, Write
Sin bajar de línea	Sin, Saltar	Without, Newline
Pantalla y espera	Borrar, Pantalla, Esperar, Segundos	Clear, Screen, Wait, Seconds
Decisión	Si, Entonces, Sino, FinSi	If, Then, Else, EndIf
Opción múltiple	Según, Hacer, De, Otro, Modo, FinSegun	Switch, Do, In, Other, Case, EndSwitch
Ciclo mientras	Mientras, FinMientras	While, EndWhile
Ciclo repetir	Repetir, Hasta, Que	Repeat, Until, That
Ciclo para	Para, Con, Paso, FinPara	For, With, Step, EndFor

Tipos	Entero, Real, Lógico, Caracter, Cadena	Integer, Real, Boolean, Character, String
Valores lógicos	Verdadero, Falso	True, False
Lógica	Y, O, NO	And, Or, Not
División entera	DIV, MOD	Div, Mod

Según, Función, FinFunción y Lógico se pueden escribir con tilde o sin ella. Caracter y FinSegun se muestran sin tilde. De Otro Modo, en inglés, se lee In Other Case. Hasta Que se lee Until That.

Los nombres de la biblioteca no se traducen: Raíz, Longitud y ConvertirATexto se escriben así en los dos idiomas. Tampoco se traducen los nombres que tú inventas, ni el texto que va entre comillas.

3. Tipos de datos

Toda variable se declara antes de usarse. Un escalar no tiene valor hasta que se le asigna uno o se lee. Si se usa antes, el programa se detiene. Las casillas de un arreglo sí nacen con un valor.

Tipo	**Para qué sirve**	**Cómo se escribe un valor**	**Casilla nueva**
Entero	Números sin parte decimal	20, -7	0
Real	Números con punto decimal	9.5, 0.0	0.0
Lógico	Una condición	Verdadero, Falso	Falso
Caracter	Un solo símbolo	'A', ' '	un espacio
Cadena	Un texto	"Ana"	texto vacío

- El real 9.5 vale exactamente 9.5. La división con / siempre da un real.
- Un entero se puede guardar en un real. Un real no se puede guardar en un entero: hay que usar Trunc o Redon.

- Un caracter se puede guardar en una cadena. Una cadena de un solo símbolo se puede guardar en un caracter.
- Un número no sirve como condición. Hay que compararlo.
- Leer un entero acepta espacios alrededor. Leer un lógico acepta Verdadero, Falso, True y False. Leer un caracter exige exactamente un símbolo.
- Si el dato no corresponde al tipo, no se guarda, no aparece en la consola y se vuelve a pedir.

```
Definir edad Como Entero;

Definir promedio Como Real;

Definir activo Como Logico;

Definir inicial Como Caracter;

Definir nombre Como Cadena;

edad <- 18;

promedio <- 9.5;

activo <- Verdadero;

inicial <- 'A';

nombre <- "Ana";
```

4. Instrucciones

Cada instrucción de abajo dice qué hace, qué hay que escribir, qué figura usa el diagrama y, cuando existe, qué bloque del laboratorio la arma. El bloque se arrastra; la instrucción es el texto que queda.

4.1 Algoritmo



Óvalo de inicio. El texto del óvalo es Inicio, o Inicio seguido del nombre si es una subrutina.

Abre el programa. Entre Algoritmo y FinAlgoritmo van las instrucciones que se ejecutan al pulsar Ejecutar.

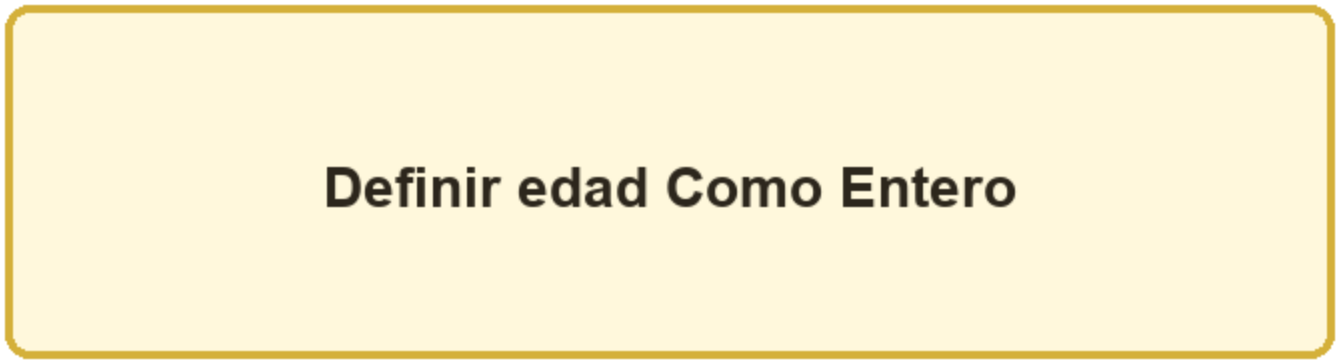
Parámetros: un nombre. No lleva paréntesis ni tipo.

```
Algoritmo AreaTriangulo

Definir base, altura, area Como Real;

FinAlgoritmo
```

4.2 Definir



Rectángulo de proceso. Declarar no cambia el valor: solo reserva el nombre y el tipo.



Bloque Definir, del grupo Variables. El primer hueco es el nombre y el segundo es el tipo.

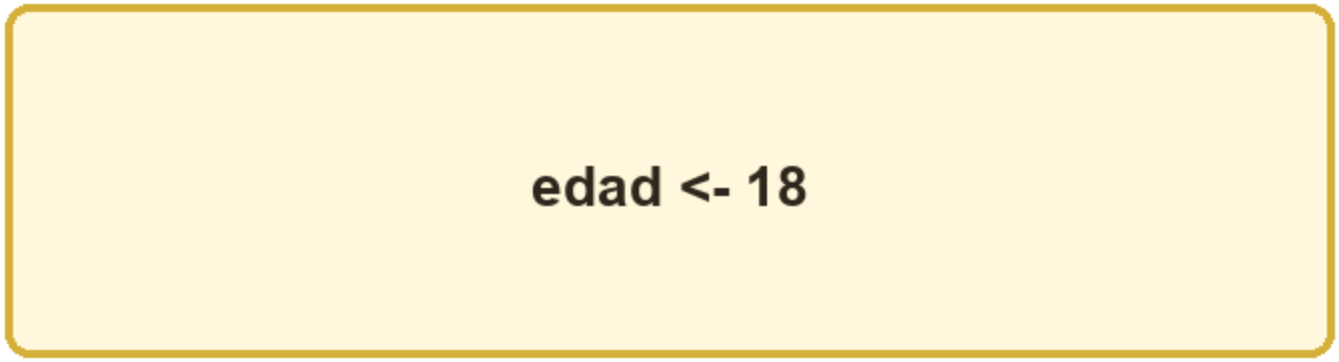
Crea una o varias variables del mismo tipo. Hay que declararla antes de leerla, asignarla o usarla en una expresión.

Parámetros: uno o más nombres separados por comas, la palabra Como y uno de estos tipos: Entero, Real, Lógico, Caracter, Cadena.

```
Definir base, altura Como Real;

Definir encontrado Como Logico;
```

4.3 Asignar



Rectángulo de proceso. La flecha <- guarda el resultado de la derecha en el nombre de la izquierda.



Bloque Asignar. A la izquierda va el nombre y a la derecha una expresión.

Guarda un valor. La expresión de la derecha se calcula primero. El tipo del resultado tiene que coincidir con el de la variable, con la única ampliación de entero hacia real.

Parámetros: un destino y una expresión. El destino es un nombre o una casilla de arreglo.

```
base <- 8;

altura <- 3.5;

area <- base * altura / 2;

encontrado <- Falso;

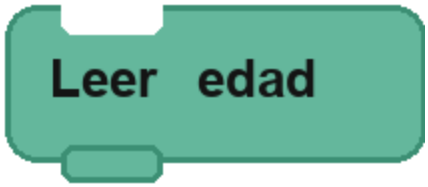
nombre <- "Ana";
```

No escribas edad = 18 para guardar. Eso compara. La forma correcta es edad <- 18.

4.4 Leer



Paralelogramo rosa. Toda entrada se dibuja con esta figura.



Bloque Leer. El hueco es el nombre de la variable que recibe el dato.

Pide un dato en la consola y detiene el programa hasta que llega. Si pides varios destinos, pide uno por cada uno, en orden. El dato aceptado se muestra en la consola. El rechazado no.

Parámetros: uno o más destinos separados por comas. Cada destino es una variable o una casilla.

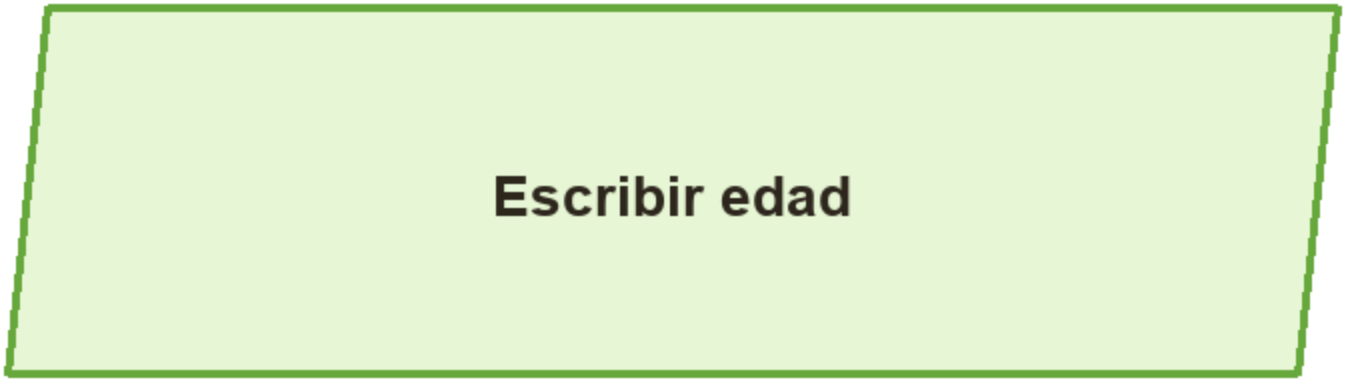
```
Escribir "Escribe tu edad";

Leer edad;

Leer nombre, inicial;
```

Lo que se acepta depende del tipo: un entero, un real con punto, Verdadero o Falso, un solo carácter, o el resto de la línea si es cadena.

4.5 Escribir



Paralelogramo verde. Toda salida se dibuja con esta figura.



Bloque Escribir. El hueco recibe el valor. La casilla «sin saltar» evita el salto de línea. El botón + agrega más valores, hasta 8.

Muestra valores en la consola. Varios valores se pegan uno tras otro, sin agregar espacios: los espacios van dentro del texto. Después baja de línea, salvo que la sentencia termine en Sin Saltar.

Parámetros: una o más expresiones separadas por comas. Opcional: Sin Saltar al final.

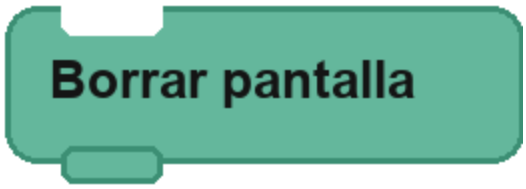
```
Escribir "Hola";
Escribir "El área es ", area;
Escribir "Nota: ", nota Sin Saltar;
Escribir " puntos";
```

Un lógico se imprime como Verdadero o Falso. Un real no muestra ceros de más ni notación científica.

4.6 Borrar Pantalla



Rectángulo de proceso. Vacía lo que se está viendo en la consola.

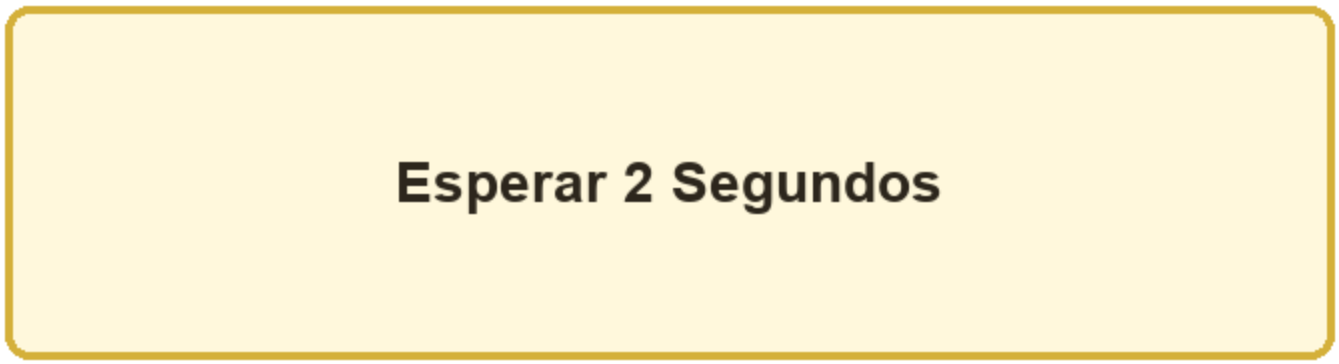


Bloque Borrar pantalla. No tiene huecos.

No recibe parámetros. Lo escrito antes deja de mostrarse. No pide confirmación.

```
Borrar Pantalla;
Escribir "Pantalla limpia";
```

4.7 Esperar



Rectángulo de proceso. El programa se detiene y luego sigue.



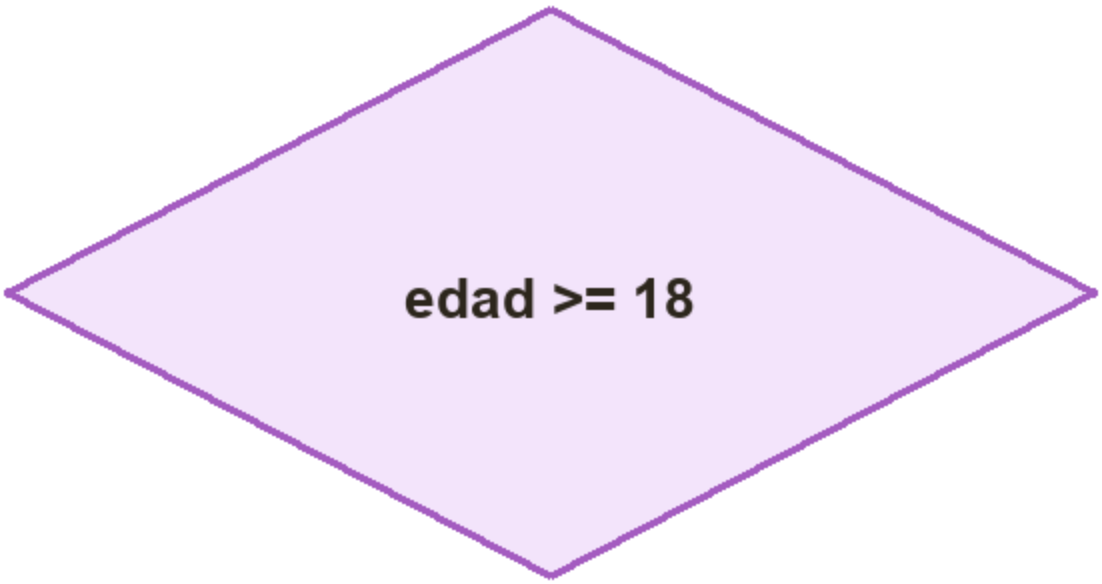
Bloque Esperar. El hueco es la cantidad de segundos, un entero o un real.

Parámetros: un número de segundos, entre 0 y 60. Ese tiempo no cuenta para el límite del ciclo.

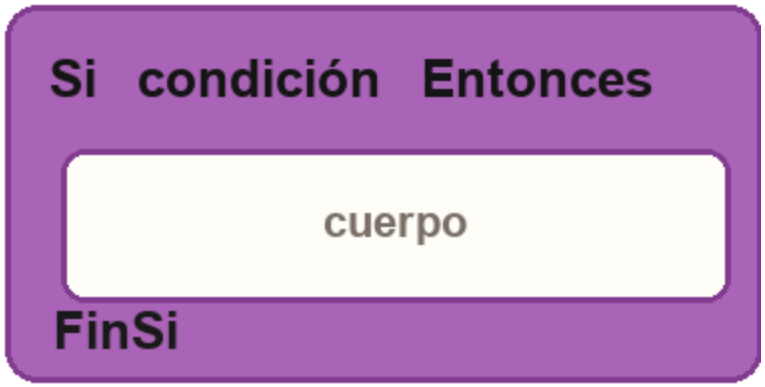
```
Escribir "Listo en dos segundos";

Esperar 2 Segundos;
```

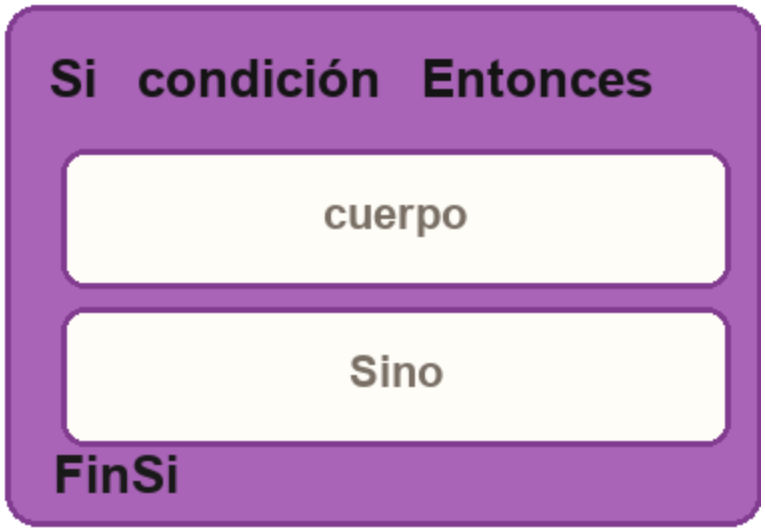
4.8 Si, Entonces, Sino y FinSi



Rombo de decisión. La rama verdadera sale por la derecha y se marca V. La falsa sale por la izquierda y se marca F. En inglés, la verdadera se marca T.



Bloque Si. La condición entra en el hueco. Las instrucciones se colocan en el espacio interior.



Bloque Si / Sino. Tiene dos espacios: el de Entonces y el de Sino.

Ejecuta un grupo de instrucciones solo cuando la condición es verdadera. Sino es opcional. Un Sino seguido de otro Si es un Si anidado: no existe la palabra SinoSi.

Parámetros: una condición de tipo lógico. Dentro van instrucciones, que pueden ser cero.

```
Si edad >= 18 Entonces

Escribir "Mayor de edad";

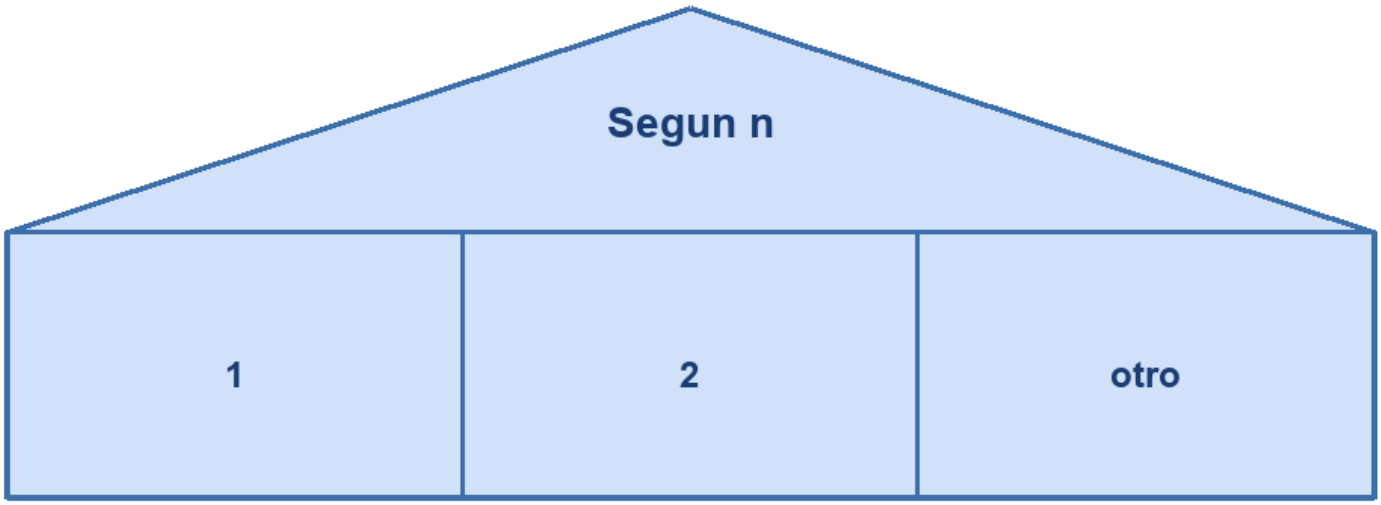
Sino

Escribir "Menor de edad";

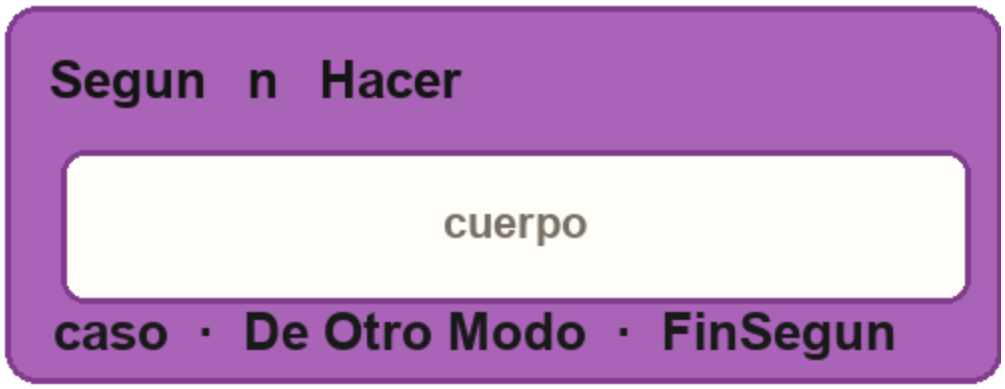
FinSi
```

La condición no puede ser un número suelto. Se escribe edad >= 18, no edad. Dos comparaciones no se encadenan: en lugar de a < b < c se escribe a < b Y b < c.

4.9 Según



Triángulo ancho con una barra de casos. El valor comparado va en el triángulo. Cada caso ocupa un tramo de la barra y De Otro Modo va al final.



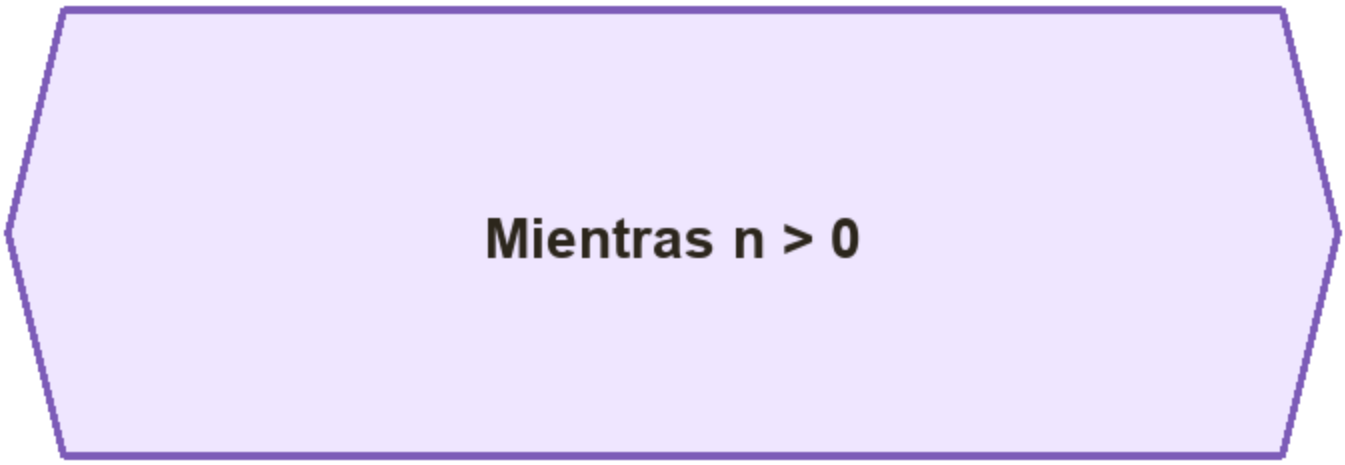
Bloque Según. El botón + agrega un caso, hasta 12. De Otro Modo se usa cuando ningún caso coincide.

Elige un solo caso. No continúa con el siguiente. Varios literales separados por comas forman un mismo caso. De Otro Modo aparece como máximo una vez y siempre al final. Si no está y ningún caso coincide, no se ejecuta nada.

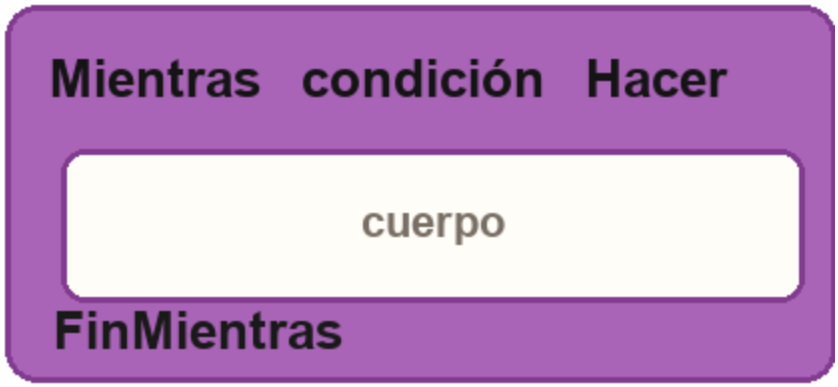
Parámetros: la expresión que se compara y, en cada caso, uno o más literales. Un literal es un número, un texto, un carácter o Verdadero o Falso. Una cuenta, como 1 + 0, no sirve de etiqueta.

```
Segun dia Hacer
1:
Escribir "Lunes";
2, 3:
Escribir "Martes o miércoles";
"fin":
Escribir "Termina";
De Otro Modo:
Escribir "Otro día";
FinSegun
```

4.10 Mientras



Hexágono de ciclo. El cuerpo queda debajo y a la derecha. La flecha vuelve al hexágono hasta que la condición es falsa.



Bloque Mientras. Primero mira la condición y, si se cumple, ejecuta el interior.

Repite el cuerpo mientras la condición sea verdadera. La mira antes de entrar, así que puede no ejecutarse nunca. No hay una palabra Salir ni Continuar: el ciclo termina cuando la condición deja de cumplirse.

Parámetros: una condición lógica.

```
n <- 3;

Mientras n > 0 Hacer

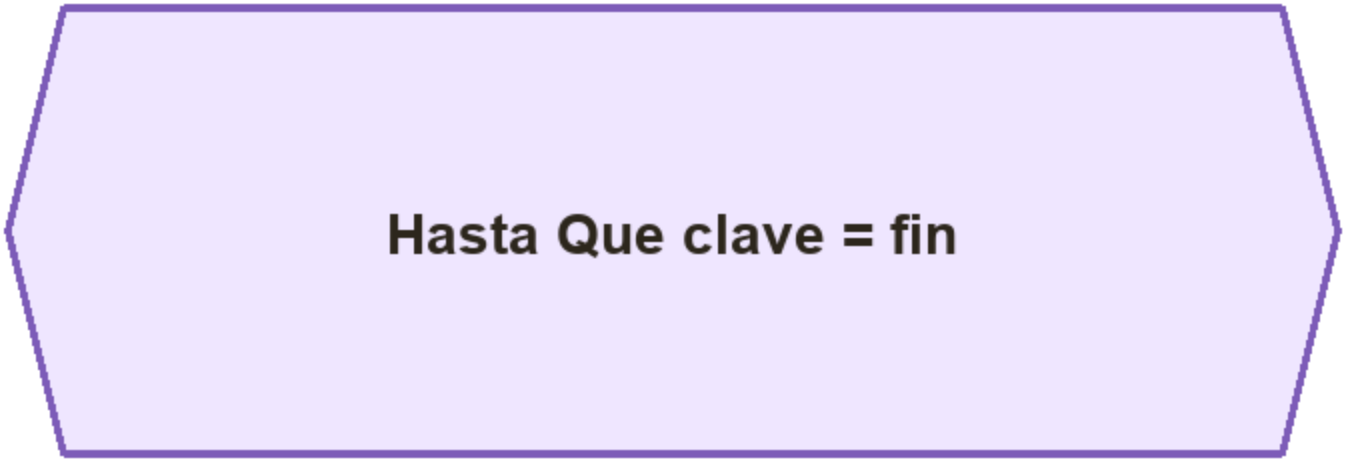
Escribir n;

n <- n - 1;

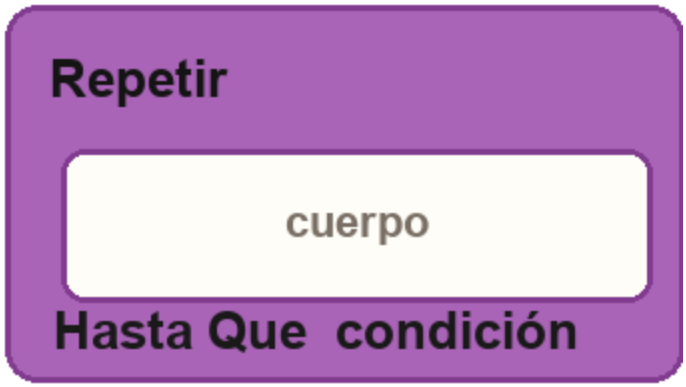
FinMientras
```

Ese programa escribe 3, luego 2 y luego 1.

4.11 Repetir



El mismo hexágono. En Repetir el cuerpo queda arriba, porque se ejecuta antes de mirar la condición.



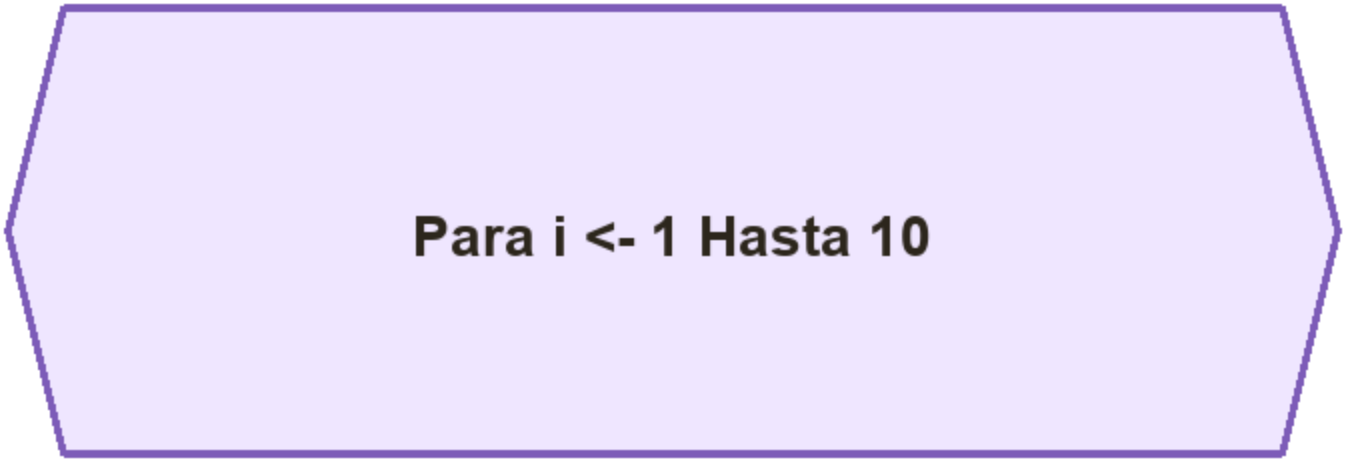
Bloque Repetir. El cuerpo va dentro y la condición va en Hasta Que.

Ejecuta el cuerpo y después comprueba. Corre al menos una vez. Termina cuando la condición queda verdadera.

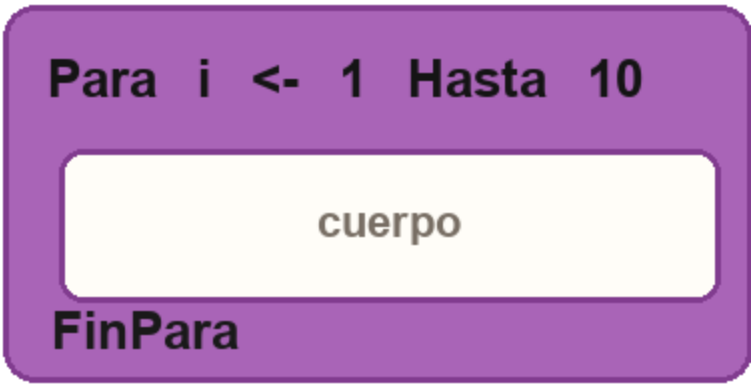
Parámetros: las instrucciones del cuerpo y una condición lógica después de Hasta Que.

```
Repetir
Escribir "Escribe fin para terminar";
Leer clave;
Hasta Que clave = "fin"
```

4.12 Para



Hexágono de ciclo. El texto resume el inicio, el límite y el paso.



Bloque Para. Pide la variable, el inicio, el límite y el paso. En el editor, Con Paso se puede omitir.

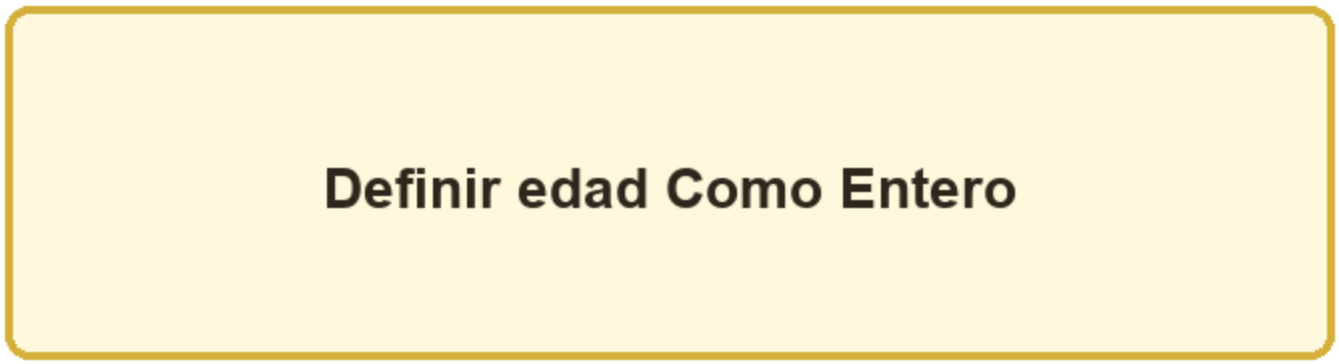
Recorre un entero. El inicio, el límite y el paso se calculan una sola vez, al entrar. Si no escribes el paso, vale 1 cuando el inicio es menor o igual que el límite, y -1 cuando el inicio es mayor. Así, de 10 hasta 1 cuenta hacia abajo. Un paso escrito como 1 no hace eso: si el inicio ya pasó el límite, el cuerpo no se ejecuta. El paso 0 es un error.

Parámetros: el nombre de un entero, el valor inicial, el valor final y, si hace falta, el paso. El nombre se declara antes.

```
Definir i Como Entero;
Para i <- 1 Hasta 5 Hacer
Escribir i;
FinPara
Para i <- 10 Hasta 1 Hacer
Escribir i;
FinPara
Para i <- 0 Hasta 10 Con Paso 2 Hacer
Escribir i;
FinPara
```

El primero escribe 1, 2, 3, 4 y 5. El segundo escribe de 10 a 1. El tercero escribe 0, 2, 4, 6, 8 y 10. Dentro del cuerpo se puede cambiar i; el paso se suma al cerrar cada vuelta.

4.13 Dimension y casillas



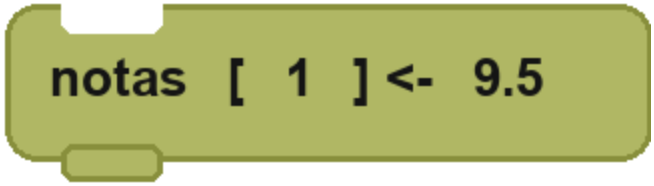
Crear el arreglo es un rectángulo de proceso. Leer o escribir una casilla usa la figura de entrada o de salida.



Bloque Dimension. El botón + agrega otra dimensión, hasta 4. Cada tamaño es el número de casillas, no el índice máximo.



Bloque Casilla. Devuelve el valor guardado. Se encaja donde hace falta una expresión.



Bloque Asignar casilla. Guarda la expresión de la derecha en esa posición.



Bloque Leer casilla. Pide un dato y lo guarda en esa posición.

El número entre corchetes es la cantidad de casillas. Con la base en 1, Dimension notas[5] crea las posiciones 1, 2, 3, 4 y 5. Con la base en 0, crea 0, 1, 2, 3 y 4. La base es del algoritmo, no de cada arreglo. Se pueden usar hasta 4 dimensiones.

Parámetros: el nombre, un tamaño por dimensión y el tipo. El tipo puede ir en el Dimension o en un Definir anterior.

```
Dimension notas[5] Como Real;

notas[1] <- 9.5;

Escribir notas[1];

Definir matriz Como Entero;

Dimension matriz[3, 3];

matriz[2, 1] <- 7;

Leer matriz[3, 3];
```

Una casilla nueva de enteros vale 0, una de reales vale 0.0, una lógica vale Falso, un caracter es un espacio y una cadena está vacía. Un índice fuera del intervalo detiene el programa e indica el intervalo válido.

4.14 Función

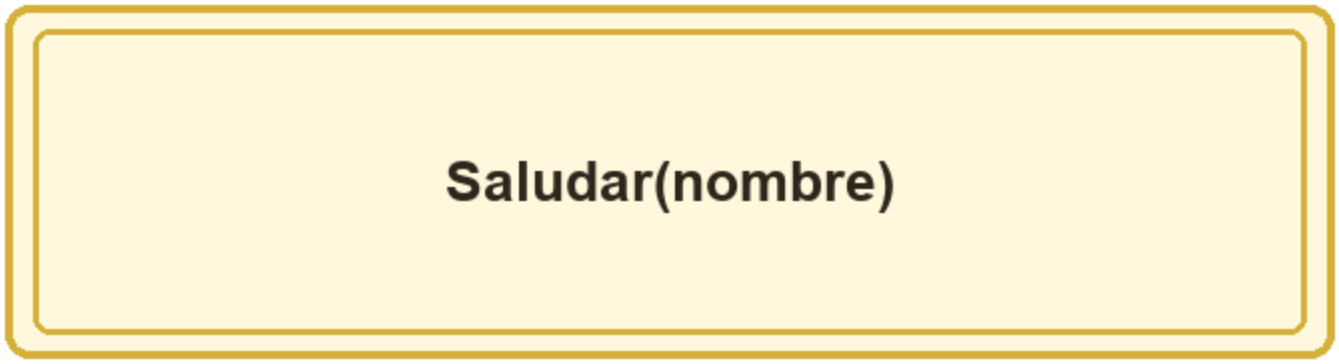


Figura de subproceso, el rectángulo con doble marco. Aparece cuando el algoritmo llama a la función.



Bloque Función. Va suelto en el lienzo, no dentro del algoritmo. El primer nombre es el resultado y el segundo es el nombre con el que se llama.



Bloque Valor de función. Devuelve el resultado y se encaja en una expresión. El tipo tiene que coincidir con lo que devuelve.

Una función calcula un valor y lo devuelve. Se escribe fuera del algoritmo. El nombre de la izquierda de <- es la variable de retorno: hay que declararla, asignarla por todos los caminos y no hace falta una instrucción especial de retorno. Al llegar a FinFunción, esa variable es el resultado.

Parámetros: el nombre del resultado, el nombre de la función y la lista de parámetros entre paréntesis. Cada parámetro es un nombre y, si quieres indicarlo, Por Valor o Por Referencia. Si no dices nada, un escalar va por valor.

```
Funcion doble <- Doblar(n Por Valor)
Definir doble Como Entero;
doble <- n * 2;
FinFuncion
Algoritmo Prueba
Definir r Como Entero;
r <- Doblar(4);
Escribir r;
FinAlgoritmo
```

Ese programa escribe 8. Doblar no ve las variables del algoritmo. Los datos entran por los parámetros y salen por doble.

4.15 SubProceso



Bloque SubProceso. También va suelto en el lienzo. No tiene variable de resultado.



Bloque Llamar procedimiento. Ocupa su propia línea. El botón + agrega argumentos, hasta 4.

Un procedimiento hace una tarea y no devuelve un valor. Se llama en su propia línea, como una instrucción. Por Valor entrega una copia. Por Referencia permite modificar la variable de quien llamó. Un arreglo, si no dices otra cosa, pasa por referencia; Por Valor hace una copia de las casillas.

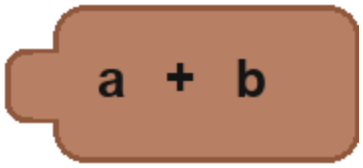
Parámetros: el nombre del procedimiento y sus parámetros. En la llamada, una expresión por cada parámetro, en el mismo orden.

```
SubProceso Saludar(nombre Por Valor)
Escribir "Hola, ", nombre;
FinSubProceso
SubProceso Incrementar(n Por Referencia)
n <- n + 1;
FinSubProceso
Algoritmo Prueba
Definir edad Como Entero;
edad <- 17;
Saludar("Ana");
Incrementar(edad);
Escribir edad;
FinAlgoritmo
```

Escribe Hola, Ana y después 18. Saludar no puede cambiar la cadena original. Incrementar sí cambia edad.

5. Operadores

Se escriben entre valores. El bloque de operación, el de comparación, el de Y / O y el de NO arman estas mismas cuentas en el laboratorio. La potencia ^ se escribe en el editor.



Bloque Operación. Acepta números. Los operadores son + - * / DIV y MOD.



Bloque Comparación. El resultado es un lógico y sirve de condición.



Bloque Y / O. Las dos partes tienen que ser lógicas.



Bloque NO. Niega una condición.

Operador	Qué hace	Ejemplo	Resultado
+	Suma	2 + 3	5
-	Resta, o cambia el signo	-2	-2
*	Multiplicación	4 * 2	8
/	División real	7 / 2	3.5
DIV	División entera, hacia cero	7 DIV 2	3
MOD o %	Residuo de esa división	7 MOD 2	1
^	Potencia	2 ^ 3	8
=	Igual	3 = 3	Verdadero
<> o !=	Distinto	3 <> 4	Verdadero
< <= > >=	Orden	3 <= 4	Verdadero
Y	Las dos condiciones	Verdadero Y Falso	Falso
O	Alguna condición	Falso O Verdadero	Verdadero
NO	Lo contrario	NO Falso	Verdadero

- La potencia liga más fuerte que el signo: -2 ^ 2 vale -4, porque es -(2 ^ 2).
- 2 ^ 3 ^ 2 vale 512, porque se lee 2 ^ (3 ^ 2).
- Y ata más fuerte que O: a O b Y c se lee a O (b Y c).
- NO a Y b se lee (NO a) Y b.

- Y, O y NO solo aceptan lógicos. Si la primera parte ya decide el resultado, la segunda no se calcula.
- DIV y MOD exigen enteros. -7 DIV 2 vale -3 y -7 MOD 2 vale -1.
- Dividir entre cero detiene el programa. 0 ^ 0 también. Una base negativa elevada a un real no entero también.
- Comparar dos reales con = puede fallar después de una división. El programa avisa y aun así se ejecuta.

6. Valores que se escriben sueltos

En el laboratorio estos bloques no son instrucciones: devuelven un valor y se encajan en un hueco. En el editor se escriben directamente.



Número entero. Sin punto decimal.



Número real. Lleva punto, como 9.5. No se usa la coma de miles.



Texto entre comillas dobles. Dentro se puede escribir \\ para una barra, \" para una comilla, \n para un salto y \t para un tabulador.



Un carácter entre comillas simples. 'AB' y '' no son válidos. El espacio se escribe ' '.



Verdadero o Falso. En inglés, True o False. El programa los acepta de las dos formas.



Bloque Variable. Entrega el valor guardado. El tipo del bloque limita los huecos donde puede entrar.



Bloque Unir texto. Pega dos cadenas. En el editor se usa la función Concatenar, que acepta de 2 a 8 valores.

7. Figuras del diagrama de flujo

El diagrama sale del algoritmo, no se dibuja a mano en el banco de trabajo. Cada figura corresponde a una clase de instrucción. La flecha es roja y entra siempre por el centro del lado de arriba. Proceso, entrada, salida, inicio, fin y llamada salen por el centro de abajo. Se puede descargar en SVG, PNG y PDF, e imprimir.

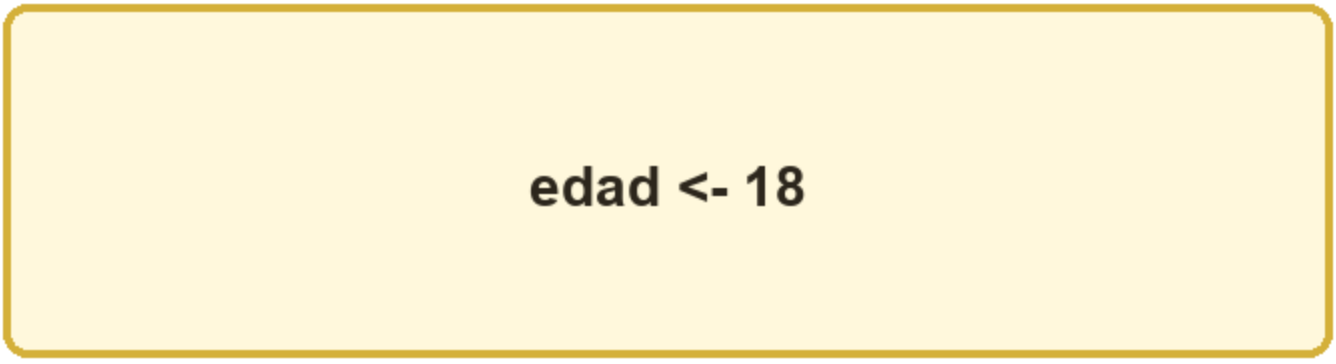
En Armar diagrama se usan estas mismas figuras al revés: las arrastras y el programa escribe el pseudocódigo.

Inicio y fin



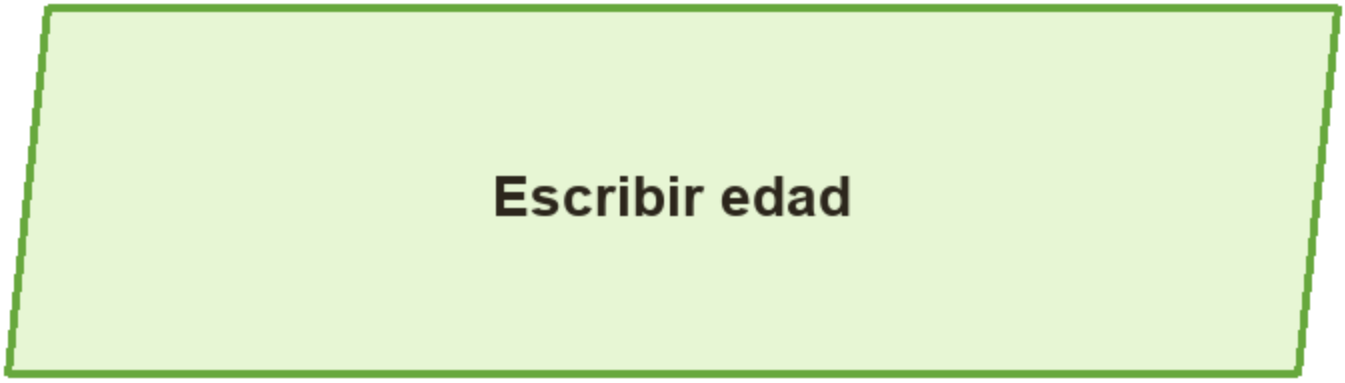
Óvalo amarillo. Marca el comienzo y el cierre del algoritmo o de una subrutina. No es una instrucción que tú escribas: aparece por Algoritmo, FinAlgoritmo, Función, FinFunción, SubProceso y FinSubProceso.

Proceso



Rectángulo crema. Sirve para Definir, Dimension, asignar, Borrar Pantalla y Esperar. El texto es la instrucción.

Salida



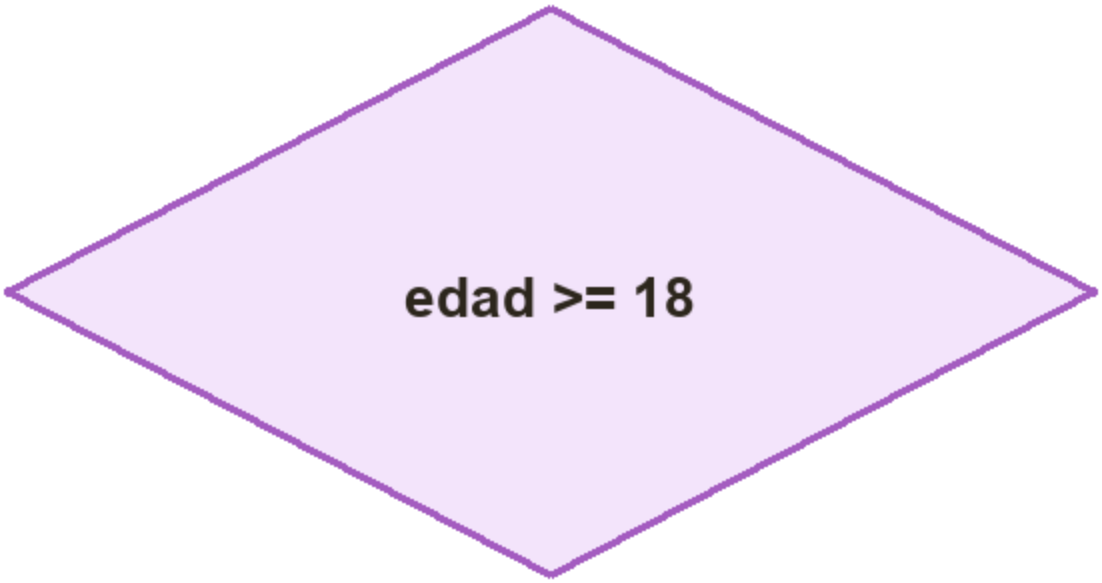
Paralelogramo verde, inclinado hacia la derecha. Solo lo usa Escribir.

Entrada



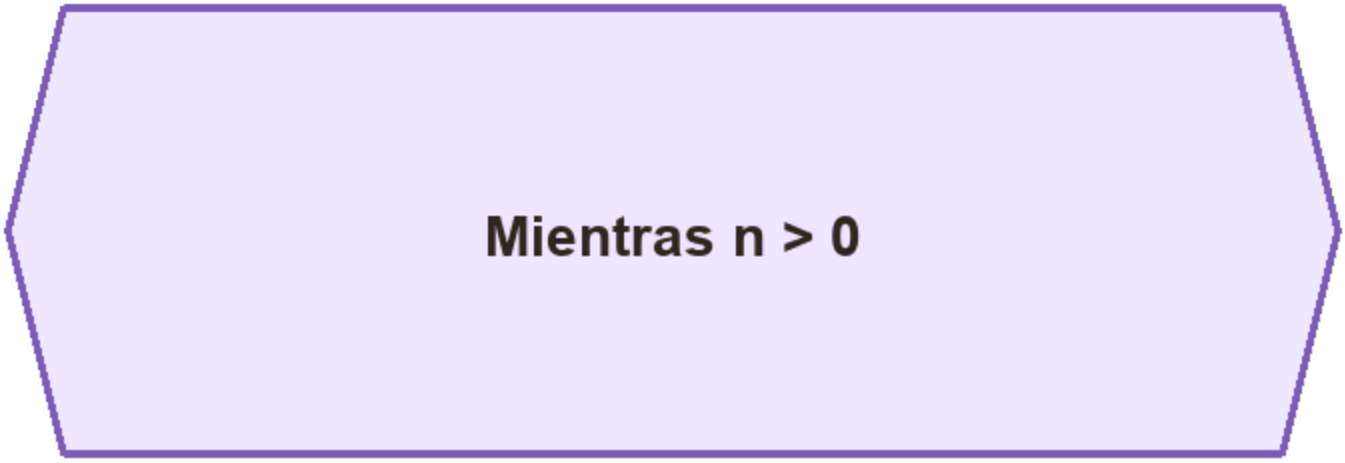
Paralelogramo rosa, inclinado hacia el otro lado. Solo lo usa Leer.

Decisión



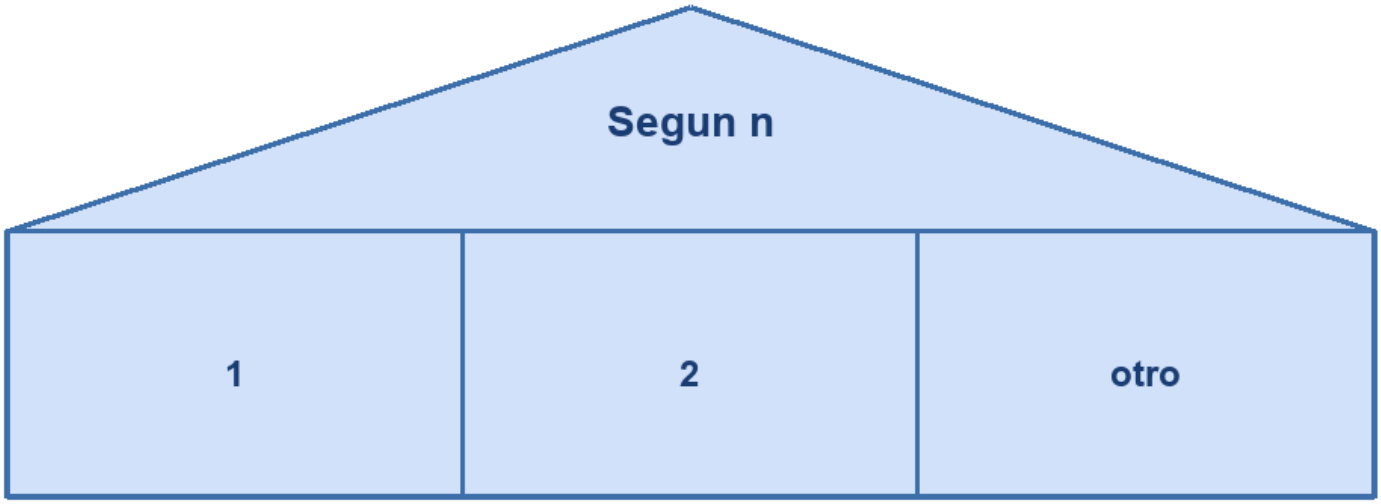
Rombo lila. Lo usa Si. La condición va dentro. La rama falsa sale por la izquierda, marcada F. La verdadera sale por la derecha, marcada V, o T si el idioma es inglés.

Ciclo



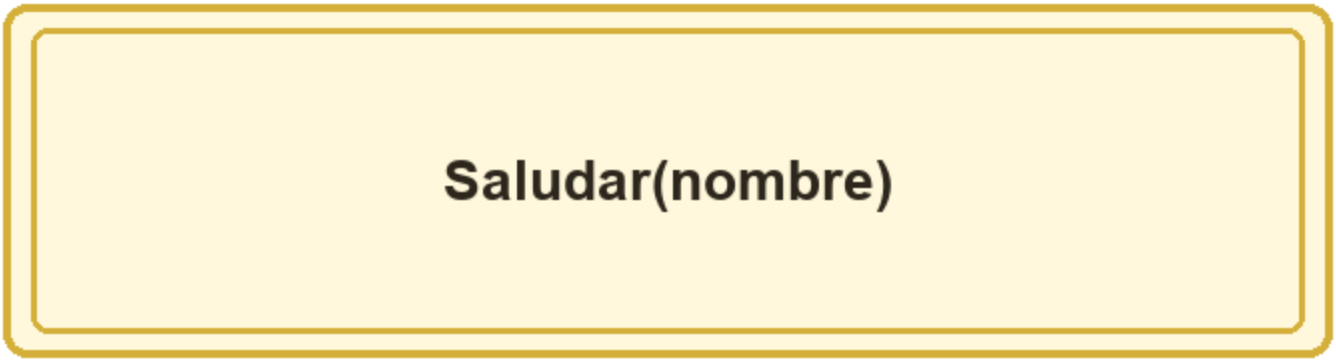
Hexágono violeta. Lo usan Mientras, Para y Repetir. En Mientras y Para el cuerpo queda abajo a la derecha. En Repetir el cuerpo queda arriba, porque ocurre antes de la pregunta.

Opción múltiple



Triángulo azul con una barra. Lo usa Según. El valor va en el triángulo y los casos en la barra, con De Otro Modo al final. No es un rombo con muchas salidas sueltas.

Llamada



Rectángulo de doble marco. Aparece al llamar una función o un procedimiento. El nombre y los argumentos van dentro.

8. Bloques del laboratorio

Los bloques están en la práctica libre y en los retos de construir. Cada grupo tiene un color. Un bloque con muesca arriba y abajo es una instrucción y se apila. Un bloque con lengüeta a la izquierda devuelve un valor y se encaja en un hueco. Una función o un procedimiento lleva la ceja superior y se deja suelto en el lienzo. El espacio en blanco de un bloque de control es el cuerpo: ahí se apilan las instrucciones de dentro.

Grupo	Color	Bloques
Variables	Azul	Definir, Asignar, Variable
Entrada y salida	Verde	Leer, Escribir, Borrar pantalla, Esperar, Unir texto
Operadores	Naranja	Entero, Real, Texto, Caracter, Verdadero o falso, Operación, Comparación, Y / O, NO
Decisiones	Morado	Si, Si / Sino, Según
Ciclos	Morado	Mientras, Para, Repetir
Arreglos	Oliva	Dimension, Casilla, Asignar casilla, Leer casilla
Funciones	Verde azulado	Función, SubProceso, Valor de función, Llamar procedimiento

Los huecos blancos son los datos que hay que completar. Si el hueco espera un número, no acepta un texto. El botón + de Escribir, Dimension, Según y de las llamadas agrega otro dato; el botón – lo quita.

Los bloques no son otro lenguaje. Al ejecutar, se convierten en el mismo pseudocódigo de este manual y lo recorre el mismo programa. Por eso un error de bloque muestra línea y columna, igual que en el editor.

9. Funciones incluidas

Se usan como una expresión: el nombre, paréntesis y los valores. No hace falta definirlas. No se pueden usar como nombre de variable. El menú Funciones del editor lista cada una. Los nombres se escriben igual en español y en inglés; lo que cambia es la explicación.

En el laboratorio, el valor de una de estas funciones se arma con el bloque Valor de función: escribes el nombre, eliges el tipo del resultado y encajas los argumentos.

Abs

Quita el signo.

Uso	Qué se le pasa	Qué devuelve
Abs(número)	Un entero o un real.	El mismo tipo, sin signo.

```
Escribir Abs(-7);
```

Escribe 7.

Raíz

Raíz cuadrada.

Uso	Qué se le pasa	Qué devuelve
Raíz(número)	Un entero o un real, cero o positivo.	Real.

```
Escribir Raíz(9);
```

Escribe 3. Un negativo detiene el programa.

Potencia

Eleva la base al exponente.

Uso	Qué se le pasa	Qué devuelve
Potencia(base, exponente)	Dos números.	Entero si los dos son enteros y el exponente no es negativo. Si no, real.

```
Escribir Potencia(2, 3);
```

Escribe 8. Potencia(2, -1) es el real 0.5.

Redon

Redondea al entero más cercano. El .5 sube.

Uso	Qué se le pasa	Qué devuelve
Redon(número)	Un entero o un real.	Entero.

```
Escribir Redon(3.5);
```

Escribe 4. Redon(3.4) escribe 3.

Trunc

Quita la parte decimal, sin redondear, hacia cero.

Uso	Qué se le pasa	Qué devuelve
Trunc(número)	Un entero o un real.	Entero.

```
Escribir Trunc(3.9);
```

Escribe 3. Trunc(-3.9) escribe -3.

Sen

Seno.

Uso	Qué se le pasa	Qué devuelve
Sen(radianes)	Un número. El ángulo va en radianes, no en grados.	Real.

```
Escribir Sen(0);
```

Escribe 0.

Cos

Coseno.

Uso	Qué se le pasa	Qué devuelve
Cos(radianes)	Un número en radianes.	Real.

```
Escribir Cos(0);
```

Escribe 1.

Tan

Tangente.

Uso	Qué se le pasa	Qué devuelve
Tan(radianes)	Un número en radianes.	Real.

```
Escribir Tan(0);
```

Escribe 0.

Ln

Logaritmo natural.

Uso	Qué se le pasa	Qué devuelve
Ln(número)	Un número mayor que cero.	Real.

```
Escribir Ln(1);
```

Escribe 0. Cero o negativo detiene el programa.

Exp

e elevado a ese número.

Uso	Qué se le pasa	Qué devuelve
Exp(número)	Un número.	Real.

```
Escribir Exp(0);
```

Escribe 1.

Aleatorio

Un número al azar.

Uso	Qué se le pasa	Qué devuelve
Aleatorio()	Ninguno.	Real entre 0 y 1, sin llegar a 1.

```
Escribir Aleatorio();
```

Cada ejecución puede mostrar un real distinto.

Aleatorio con intervalo

Un entero al azar dentro del intervalo.

Uso	Qué se le pasa	Qué devuelve
Aleatorio(inicio, fin)	Dos enteros. El primero menor o igual que el segundo.	Entero, incluidos los dos extremos.

```
Escribir Aleatorio(1, 6);
```

Escribe un entero del 1 al 6, como un dado.

Longitud

Cuenta los símbolos.

Uso	Qué se le pasa	Qué devuelve
Longitud(cadena)	Una cadena o un caracter.	Entero.

```
Escribir Longitud("Ana");
```

Escribe 3.

Subcadena

Recorta el texto. Los índices siguen la base del algoritmo.

Uso	Qué se le pasa	Qué devuelve
Subcadena(cadena, desde, hasta)	Una cadena y dos enteros. Los dos extremos entran en el recorte.	Cadena.

```
Escribir Subcadena("Traza", 1, 3);
```

Con la base en 1 escribe Tra. Con la base en 0, el 1 ya no es la primera letra.

Concatenar

Los une, convirtiéndolos a texto, sin agregar espacios.

Uso	Qué se le pasa	Qué devuelve
Concatenar(a, b, …)	De 2 a 8 valores de cualquier tipo.	Cadena.

```
Escribir Concatenar("Nota: ", 10);
```

Escribe Nota: 10.

Mayúsculas

Pasa el texto a mayúsculas.

Uso	Qué se le pasa	Qué devuelve
Mayúsculas(cadena)	Una cadena o un caracter.	Cadena.

```
Escribir Mayúsculas("ana");
```

Escribe ANA.

Minúsculas

Pasa el texto a minúsculas.

Uso	Qué se le pasa	Qué devuelve
Minúsculas(cadena)	Una cadena o un caracter.	Cadena.

```
Escribir Minúsculas("ANA");
```

Escribe ana.

ConvertirANumero

Interpreta el texto como número.

Uso	Qué se le pasa	Qué devuelve
ConvertirANumero(cadena)	Un texto con un entero o con un real que use punto.	Entero si no hay punto. Real si hay punto.

```
Escribir ConvertirANumero("18");
```

Escribe 18. ConvertirANumero("3.5") da el real 3.5. Un texto vacío o con letras detiene el programa.

ConvertirATexto

Escribe el valor como texto para poder unirlo o recortarlo.

Uso	Qué se le pasa	Qué devuelve
ConvertirATexto(valor)	Cualquier valor.	Cadena.

```
Escribir ConvertirATexto(18);
```

Devuelve "18". No agrega comillas.

10. Un programa completo

Lee tres notas en un arreglo, calcula el promedio con una función y dice si se aprobó. El punto y coma va en las sentencias simples. Los índices empiezan en 1.

```
Funcion promedio <- Media(notas Por Referencia, cantidad Por Valor)
Definir promedio Como Real;
Definir i Como Entero;
Definir suma Como Real;
suma <- 0;
Para i <- 1 Hasta cantidad Hacer
suma <- suma + notas[i];
FinPara
promedio <- suma / cantidad;
FinFuncion
Algoritmo Notas
Definir notas Como Real;
Definir i Como Entero;
Definir resultado Como Real;
Dimension notas[3];
Para i <- 1 Hasta 3 Hacer
Escribir "Nota ", i;
Leer notas[i];
FinPara
resultado <- Media(notas, 3);
Escribir "Promedio: ", resultado;
Si resultado >= 6 Entonces
Escribir "Aprobado";
Sino
Escribir "No aprobado";
```

```
FinSi

FinAlgoritmo
```

El diagrama de este programa tiene un óvalo de inicio, un hexágono para cada Para, un paralelogramo rosa en cada Leer, un paralelogramo verde en cada Escribir, un rectángulo de doble marco en la llamada a Media y un rombo para el Si. La rama Aprobado sale por la derecha.

11. Qué conviene revisar cuando no corre

- Falta el punto y coma en una sentencia simple, o sobra uno vacío.
- Se usó = para guardar. Hay que usar <-.
- La variable no está declarada, está declarada dos veces o se usó antes de darle valor.
- La condición es un número. Hay que compararlo.
- Falta FinSi, FinMientras, FinPara, FinSegun o FinAlgoritmo.
- El índice del arreglo está fuera del intervalo, o se contó desde 0 cuando el algoritmo empieza en 1.
- La etiqueta de Según es una cuenta y no un literal.
- Una función llega al final sin haber asignado su resultado.
- DIV o MOD reciben un real, o el divisor es cero.
- Leer recibió un texto que no corresponde al tipo. Se vuelve a pedir; no hace falta reiniciar si el dato siguiente es válido.

El mensaje indica la línea y la columna. En el editor puedes pulsar el diagnóstico para ir a ese lugar. En el laboratorio, el mismo mensaje señala el bloque.

12. Traducir el algoritmo

El botón Traducir está en la barra del banco de trabajo. Pide el código del algoritmo abierto en otros lenguajes de programación. Usa el mismo árbol que Validar. No ejecuta el programa y no cambia el pseudocódigo guardado.

Si el algoritmo tiene errores, no aparece código: hay que corregirlos. Una advertencia no lo impide. Los lenguajes son Python, JavaScript, TypeScript, Java, C, C++ y C#. Cada uno va en una pestaña. Ampliar agranda el recuadro y Copiar deja el texto en el portapapeles.

Este algoritmo:

```
Algoritmo Elegir

Definir n Como Entero;

n <- 2;

Segun n Hacer

1: Escribir "uno";

2, 3: Escribir "varios";

De Otro Modo:

Escribir "otro";

FinSegun

FinAlgoritmo
```

En Python el Según sale como match, con case 2 | 3. En los otros seis lenguajes sale como switch, con case 2, case 3, break y default. El resultado de ese ejemplo es varios.

12.1 Qué se conserva

- La estructura: Si, Según, Mientras, Repetir, Para, funciones y procedimientos.
- Los comentarios, de línea y de bloque.
- La base de índices del algoritmo. Si empieza en 1, el código generado también cuenta desde 1.
- DIV y MOD hacia cero. -7 DIV 2 sigue valiendo -3 y -7 MOD 2 sigue valiendo -1.
- Un parámetro por referencia. En C es un puntero, en C++ una referencia y en C# va con ref. En Python, JavaScript, TypeScript y Java va dentro de una caja, para poder cambiar el original.
- La biblioteca. Abs, Raíz, Longitud y el resto dan el mismo resultado que en Traza, escritos en la forma de cada lenguaje.
- Los nombres que inventas. Si chocan con una palabra del lenguaje de destino, se les añade un sufijo para que el programa compile.

12.2 Según

Según no siempre cabe en un switch. Cuando no cabe, sale una cadena de Si, y el resultado es el mismo.

- Python usa match con el tipo que sea.
- Un entero o un caracter caben en el switch de JavaScript, TypeScript, Java, C, C++ y C#.
- Una cadena cabe en JavaScript, TypeScript, Java y C#. En C y C++ se compara el texto, porque su switch no acepta cadenas.
- Un real cabe en Python, JavaScript y TypeScript. En Java, C, C++ y C# sale como Si.

- Un lógico cabe en todos menos en Java, que sale como Si.
- Cada caso termina para no caer en el siguiente. De Otro Modo es el caso por defecto. Si no hay De Otro Modo y ningún caso coincide, no se hace nada.

12.3 Ciclos

- Mientras sale como while. Si la condición es falsa desde el principio, el cuerpo no se ejecuta.
- Repetir sale como do y while en los lenguajes que lo tienen. La condición se niega, porque Hasta Que para cuando se cumple. El cuerpo corre al menos una vez.
- Python no tiene do while. Repetir sale como while True y un break cuando la condición se cumple.
- Para incluye el valor inicial y el final. Para i <- 1 Hasta 3 entra con 1, 2 y 3.
- Si el paso es positivo, se sigue mientras la variable sea menor o igual que el final. Si es negativo, mientras sea mayor o igual. El paso 1 suma uno y el paso -1 resta uno.
- Si el paso no es un número escrito en el fuente, el código mira en cada vuelta si es positivo o negativo.
- En Python se usa range. El extremo se desplaza para que el último valor entre: Hasta 3 con paso 1 llega hasta 3, no se queda en 2.

12.4 Dónde se genera

En el navegador, Traducir pide el código al servidor. En la aplicación de escritorio lo genera la propia máquina, con el mismo núcleo. En los dos casos el pseudocódigo guardado no se modifica y el programa generado no se ejecuta desde Traza.

Actualizar, en la misma barra, vuelve a leer proyectos y algoritmos del servidor. Sirve cuando la misma cuenta está abierta en otra ventana. En esta ventana suelta lo que todavía no se guardó.

12.5 Si el docente la apaga

En Comunidades, el docente puede apagar la traducción para todo el grupo o solo para un alumno. Si está apagada, Traducir no muestra código y avisa que el docente la desactivó. El pseudocódigo se sigue escribiendo, validando y ejecutando igual.

Repaso, en el banco de trabajo del docente, abre los proyectos del alumno para verlos. No los modifica y no crea una versión al ejecutarlos.

12.6 Pegar y el nivel de la cuenta

Pegar no cambia el lenguaje. Si el docente lo apaga para el grupo o para un alumno, el editor deja escribir y no acepta texto pegado. La misma regla vale si el nivel de la cuenta no incluye pegar.

El nivel también puede apagar Traducir. En ese caso el botón no genera código, aunque la comunidad lo tenga habilitado. El nivel gratis de alumno no incluye traducir, el diagrama, depurar ni descargar. Esas funciones están en el nivel premium, y el administrador decide el precio y si el navegador puede abrir la aplicación.

Si el navegador no está permitido, al entrar solo se ve la cuenta: el tipo, la foto, la contraseña y los días que faltan. El editor, Traducir y el laboratorio se usan en la aplicación de escritorio. La foto es JPG o PNG, de hasta 120 KB. Cambiar la contraseña pide la actual.